

Mastermind + Wordle — Tests

Project 2, Phase II · tests/assignment2/ · EID ak54582

Wordle tests, Mastermind regression tests, and results of the latest run

How the tests run

The suites have no dependencies (no JUnit), so they run anywhere a JDK is installed. Each exits with status 0 only if every check passes.

```
javac -d out src/assignment2/*.java tests/assignment2/*.java
java -cp out assignment2.AllTests          # both suites
java -cp out assignment2.WordleTests       # Wordle only
java -cp out assignment2.MastermindTests   # Mastermind: Phase I tests + Phase II regression
```

As in Phase I, tests work at three levels: pure scoring, the game API (with a `FixedSecretSource`, so the secret is known), and full console sessions fed through the **same** `ConsoleRunner` with an injected reader and writer. Phase II adds a fourth level: the real launcher, `java assignment2.Driver ...`, run as a child process. The helpers (`check`, `session`, `launch`, `allCodes`) moved into a shared `TestSupport` class, so both suites use one harness.

Game-level Wordle tests use a 20-word list built in the test, so each expected value can be checked by hand. The "real word file" and launcher tests load the external `data/words.txt` exactly as the game does.

220 / 220

checks passed

129

Wordle checks

91

Mastermind checks (80 Phase I + 11 new)

6 / 6

planted bugs caught

Latest run: 220 passed, 0 failed, about 7 seconds. It passes both compiled with JDK 17 (`--release 8`) and on a real JDK 8 (1.8.0_492).

Required: repeated-letter cases

The assignment requires two kinds of repeated-letter test: **(a)** the guess has more copies of a letter than the secret, and **(b)** one occurrence is CORRECT while another occurrence of the same letter must **not** be marked PRESENT. All of these are checks in `WordleTests.scoringRepeatedLetters()`.

Secret	Guess	Feedback	Kind	Why this answer
APPLE	ALLEY	C P A P A	(a)	Example from the assignment. Two L's guessed, one L in the secret: only the first L is P, the second is A.
STEAM	EERIE	P A A A A	(a)	Three E's guessed, one in the secret and none in place: only the first E is P.
LEVEL	EERIE	P C A A A	(a)	Secret has two E's, guess has three: one C, one P, and the third E is A.
WORLD	HELLO	A A A C P	(b)	The only L in WORLD is matched C at position 4, so the L at position 3 must be A, not P.
THOSE	GEESE	A A A C C	(b)	The secret's single E is C at the end; the two earlier E's get nothing.
CRANE	EERIE	A A P A C	(a)+(b)	The C match is the <i>last</i> E but is assigned first, so the two earlier E's are A. A one-pass left-to-right scorer would wrongly give P to the first E.
ROBOT	FLOOR	A A P C P	(b)	One O is C; the other O is P only because the secret has a second O.

SPEED	ERASE	P A A P P	repeat in both	Both words have two E's, none in place, so both E's are P.
APPLE	PLANE	P P P A C	repeat in secret	Secret has two P's, guess has one: that P is P once.
HELLO	LLAMA	P P A A A	repeat in both	Secret has two L's, so both guessed L's are P.
ABBEY	BABES	P P C C A	(b)	One B is C; the other B is P via the secret's second B.
KAYAK	KNACK	C A P A C	repeat in both	Both K's are C; one A guessed against two in the secret.
EERIE	EERIE	C C C C C	repeat in both	All repeated letters in place: solved.

Beyond the hand-picked cases, the rule is also tested as a property on 200,000 random pairs with heavy repetition: for every letter, $\#C + \#P$ equals $\min(\text{copies in secret}, \text{copies in guess})$. That is exactly "a secret letter can satisfy at most one guessed letter, and no possible match is missed".

Wordle test cases

W1 Every secret × every guess against an independent reference SCORING

Setup	A reference scorer written separately (nested loops with "used" flags, no maps), as in Phase I's Mastermind tests.
Input	Every pair of strings over: 3 letters × length 5 (59,049 pairs); 5 letters × length 4 (390,625 pairs); 2 letters × length 7 (16,384 pairs). This covers every repeat pattern for those sizes. Plus 200,000 random pairs of length 1–10.
Expected	0 mismatches. Also: C exactly where letters are equal; P never given for a letter the secret lacks; solved exactly when guess = secret; Wordle's #C/#P equal Mastermind's black/white for the same strings.
Why	The hand cases above can only cover a few patterns; this covers all of them for small alphabets.
Result	PASS

W2 Invalid guesses are rejected and never use an attempt (API) INVALID INPUT

Setup	Default configuration (6 guesses, 5 letters), secret APPLE
Input	<code>""</code> , <code>APP</code> , <code>APPLES</code> , <code>APPL3</code> , <code>AP LE</code> , <code>ZZZZZ</code> (not a word), five spaces, tab, <code>ÄPPLE</code> , <code>apple!</code> , <code>A.P.P</code> ; then <code>__alley__</code>
Expected	All 11 throw <code>IllegalArgumentException</code> from <code>submit</code> ; <code>attemptsUsed()</code> stays 0 and history stays empty. The messages name the problem: "exactly 5 letters", "only the letters A-Z", "ZZZZZ is not in the word list". The padded lowercase <code>alley</code> is accepted, stored as <code>ALLEY</code> , scored <code>C P A P A</code> , and uses exactly one attempt. A later invalid guess changes nothing.
Result	PASS

W3 Invalid guesses in a console session INVALID INPUT

Setup	Secret APPLE, normal mode
Input	<code>APP</code> , <code>ZZZZZ</code> , <code>APP1E</code> , empty line, <code>APPLE</code> , <code>N</code>

Expected	Exactly 4 <code>INVALID_GUESS</code> lines, including <code>INVALID_GUESS: ZZZZZ is not in the word list</code> . "Guess 1 of 6" appears 5 times because the counter does not advance. Then a win.
Result	PASS

W4 HISTORY HISTORY

Setup	Secret APPLE
Input	<code>HISTORY</code> , <code>ALLEY</code> , <code>CRANE</code> , <code>HISTORY</code> , <code>history</code> , <code>APPLE</code> , <code>N</code> ; separately <code>ALLEY</code> , <code>ZZZZZ</code> , <code>HISTORY</code>
Expected	The first <code>HISTORY</code> prints "No guesses yet." Later ones list 1. <code>ALLEY C P A P A</code> before 2. <code>CRANE A A P A C</code> . The lowercase command also works (2 history blocks). The next prompt is "Guess 3 of 6", so <code>HISTORY</code> costs no attempt. An invalid guess never appears in the history.
Result	PASS

W5 Win, loss, and the last guess WIN / LOSS

Setup	Secret APPLE, 6 guesses
Input	(a) <code>APPLE</code> at once. (b) <code>6 × CRANE</code> . (c) <code>5 × CRANE</code> , then <code>APPLE</code> .
Expected	(a) <code>WON</code> , solved, feedback <code>C C C C C</code> ; a further <code>submit</code> throws <code>IllegalStateException</code> . In a console session, <code>ALLEY</code> then <code>APPLE</code> prints "You win! Solved in 2 guess(es)." (b) Still <code>IN_PROGRESS</code> after 5; <code>LOST</code> after the 6th; "You lose! The secret was APPLE."; no 7th prompt. (c) <code>WON</code> , not <code>LOST</code> .
Result	PASS

W6 Another game, and testing mode with fixed secrets REPLAY / TEST MODE

Setup	Test mode, fixed secrets APPLE then CRANE
Input	<code>APPLE</code> , <code>Y</code> , <code>ALLEY</code> , <code>HISTORY</code> , <code>CRANE</code> , <code>N</code>
Expected	"[TEST MODE] Secret: APPLE" is printed before "Secret: CRANE" (the first secret is not skipped, as pinned by a Phase I regression test). Two wins. Round 2's history shows only 1. <code>ALLEY P A A P A</code> . In normal mode the secret is never printed.
Why	The assignment requires deterministic or revealed secrets in testing mode; this covers both.
Result	PASS

W7 Bad secrets are rejected before play TEST MODE

Input	Fixed secret <code>ZZZZZ</code> (not in the list), <code>APPLES</code> (6 letters), <code>_apple_</code> ; Driver with <code>-Dwordle.secret=APPLE,QQQQQ</code>
Expected	The first two throw <code>IllegalStateException</code> when the round starts; <code>apple</code> is normalized to <code>APPLE</code> . The Driver rejects <code>QQQQQ</code> at startup even though it is the <i>second</i> entry.
Result	PASS

W8 The word list is external data WORD LIST

Input	(a) Entries <code>_apple_</code> , <code>Crane</code> , blanks, <code># comment</code> , <code>#APPLE</code> , a duplicate <code>APPLE</code> , <code>apples</code> , <code>ap-le</code> , <code>12345</code> , <code>ÄPFEL</code> , <code>ALLEY→</code> , <code>null</code> . (b) A temp file in UTF-8 with a byte-order mark and CRLF line endings. (c) A missing file. (d) The real <code>data/words.txt</code> .
-------	--

Expected	(a) Exactly APPLE, CRANE, ALLEY, in file order; words() is unmodifiable; a list with no 5-letter words is rejected. (b) APPLE, CRANE. (c) IOException, which the Driver turns into "Error: word list not found: ... (run from the project folder, or set -Dwordle.words=PATH)" and exit code 1. (d) Found, 2,271 words, every entry exactly 5 letters A–Z, no duplicates, and the example words present.
Result	PASS

W9 Changed configuration CONFIGURATION

Setup	(a) 4-letter list COLD CORD CARD WARD WORD, 3 guesses, secret WORD. (b) 2 guesses, secret APPLE. (c) -Dwordle.words=<4-letter file> -Dwordle.length=4 through the Driver.
Input	(a) APPLE, CARD, CORD, WORD. (b) CRANE, ALLEY.
Expected	(a) Rules mention "4-letter word" and "3 guesses"; the 5-letter guess is invalid; A A C C, A C C C; "Solved in 3 guess(es)". (b) Lost after exactly 2 valid guesses. (c) cord valid, APPLE invalid. The defaults are 6 and 5; 0 or negative values are rejected; a 5-letter list with a 4-letter game is rejected.
Why	Shows that nothing in the game logic hard-codes 5 letters or 6 guesses.
Result	PASS

W10 Both games from the same program (real launcher) LAUNCHER

Setup	Child JVMs running java assignment2.Driver ... on the test class path (each killed if not finished within 60 s)
Input	wordle test with -Dwordle.secret=APPLE; WORDLE_SECRET=CRANE; wordle in normal mode; mastermind test with -Dmastermind.secret=BGOP; Wordle started from inside out/; missing word file; secret not in the list; no arguments; tetris
Expected	Secret revealed, Result: C P A P A, history and win, exit 0; the environment variable works; normal mode never prints the secret; Mastermind still runs from the same program; data/words.txt is found from out/; the error cases print one Error: line, no stack trace, exit 1; the usage message names both games.
Result	PASS

W11 Fuzz, end of input, and every word as the secret ROBUSTNESS

Input	100,000 random strings through validate / submit (19,846 valid, 80,154 invalid); 20,000 random console lines including HISTORY, Y, N and non-ASCII text; a 1,000,000-character line; input ending mid-game, at the replay prompt, or empty; CRLF line endings. A solver then plays all 2,271 words in data/words.txt as the secret through the real game object, always guessing the first candidate consistent with the feedback so far.
Expected	validate()==null exactly when submit succeeds, and attempts change only on success. No exception reaches the user. For the solver, the real secret is never ruled out (the game's feedback always agrees with the independent reference), and every game ends WON or LOST within 6 guesses. Measured: 2,220 of 2,271 won (97.8%), 4.27 guesses on average when won.
Result	PASS

Mastermind regression tests

R1 Every Phase I check is kept REGRESSION

Setup	All 80 checks from Phase I's MastermindTests, updated only for renamed types (MastermindConfiguration, FixedSecretSource, RandomCodeSource).
-------	--

Expected	Same results as in Phase I, including the exhaustive comparison of every secret with every guess (1,679,616 + 1,048,576 pairs), 200,000 random property cases, and the solver's measured values: all 1,296 secrets won, worst case 9 guesses on RYRP, average 5.765. These numbers match Phase I exactly.
Why	<code>MastermindScorer</code> now uses the shared <code>PositionMatcher</code> , and these checks compare it against an independent reference on every possible pair.
Result	PASS (80/80)

R2 Console output is identical to the Phase I build REGRESSION

Setup	Before any Phase II change, the Phase I build (from <code>Project2_Phase1_ak54582.zip</code>) was run on two scripted sessions and the complete output was saved in the test.
Input	(a) Test mode, secrets BGOP then RRGG: HISTORY, XXXX, BG, rrrr, YBGO, history, BGOP, maybe, Y, RGRG, RRGG, N. (b) 3 guesses, 6 pegs, digit colors, secret 012345, ending in a loss.
Expected	The Phase II output equals the saved Phase I output character for character (line endings normalized): banner, rules, prompts, error messages, feedback, history, win/loss and replay text.
Result	PASS

R3 Mastermind through the new Driver REGRESSION

Input	<code>createGame("mastermind")</code> and <code>"MASTERMIND"</code> ; <code>-Dmastermind.secret=BGOP, rrgg; BGOP,BGOX; BGOP,,RRRR; -Dwordle.secret=APPLE</code> with Mastermind; a fixed secret <code>BGO</code> ; <code>score("BGOP","BGO")</code> ; <code>validate(null)</code>
Expected	Phase I defaults (12 guesses, 4 pegs); case-insensitive name; secrets used in order and normalized (<code>rrgg</code> → <code>RRGG</code>); a bad later entry or an empty entry is rejected at startup; Wordle's test secret does not leak into Mastermind; wrong-length secret and mismatched scorer lengths are rejected; null input still rejected and whitespace still trimmed.
Result	PASS

Do the tests catch real bugs? Planted-bug check

To check that the tests are strong enough, one deliberate bug at a time was planted in a scratch copy of the code, and both suites were run against it. A good suite should fail on every planted bug.

Planted bug	Wordle checks failed	Mastermind checks failed	Caught?
M1 PRESENT whenever the letter appears anywhere in the secret (no counting)	18	10	yes
M2 A PRESENT match does not use up the secret letter	15	6	yes
M3 CORRECT matches are not removed before PRESENT is assigned	11	9	yes
M4 PRESENT assigned right to left instead of left to right	14	0	yes
M5 A secret is no longer checked when a round starts	2	2	yes
M6 Wordle accepts any 5 letters (word list not checked)	12	0	yes

M4 does not change Mastermind's black/white counts (only *which* repeated letter gets P), so only the Wordle tests can catch it, and they do. While running this check, one Wordle test crashed instead of failing (it called `.equals` on a `null` message); it was rewritten so that a broken `validate()` produces a FAIL line.

All checks by group

Suite	Group (method)	What it covers	Checks	Result
Wordle	scoringRepeatedLetters	Required repeated-letter cases (table above)	13	13/13
Wordle	scoringOtherCases	All correct/absent, anagram, format, guards, value semantics	10	10/10
Wordle	scoringExhaustiveVsOracle	Every pair over small alphabets (W1)	3	3/3
Wordle	scoringProperties	Per-letter rule, 200,000 random pairs, cross-check with Mastermind (W1)	6	6/6
Wordle	wordList	Dictionary file format and loading (W8)	8	8/8
Wordle	realWordFile	data/words.txt itself (W8)	5	5/5
Wordle	configuration	Defaults, invalid settings, -D properties (W9)	8	8/8
Wordle	gameRules	Invalid input, win/loss, history, reset, fixed and random sources (W2, W5, W7)	25	25/25
Wordle	consoleSessions	Scripted sessions through the shared ConsoleRunner (W3–W6)	23	23/23
Wordle	changedConfiguration	4 letters / 3 guesses; 2 guesses (W9)	5	5/5
Wordle	driverAndLauncher	Driver in-process and real launcher processes (W7, W10)	17	17/17
Wordle	fuzz	Random and oversized input (W11)	4	4/4
Wordle	solverEveryWord	All 2,271 words played (W11)	2	2/2
Mastermind	Phase I groups (9 methods)	Unchanged Phase I suite (R1)	80	80/80
Mastermind	phase2Regression	Golden transcripts, Driver, test secrets (R2, R3)	11	11/11
Total			220	220/220